2026

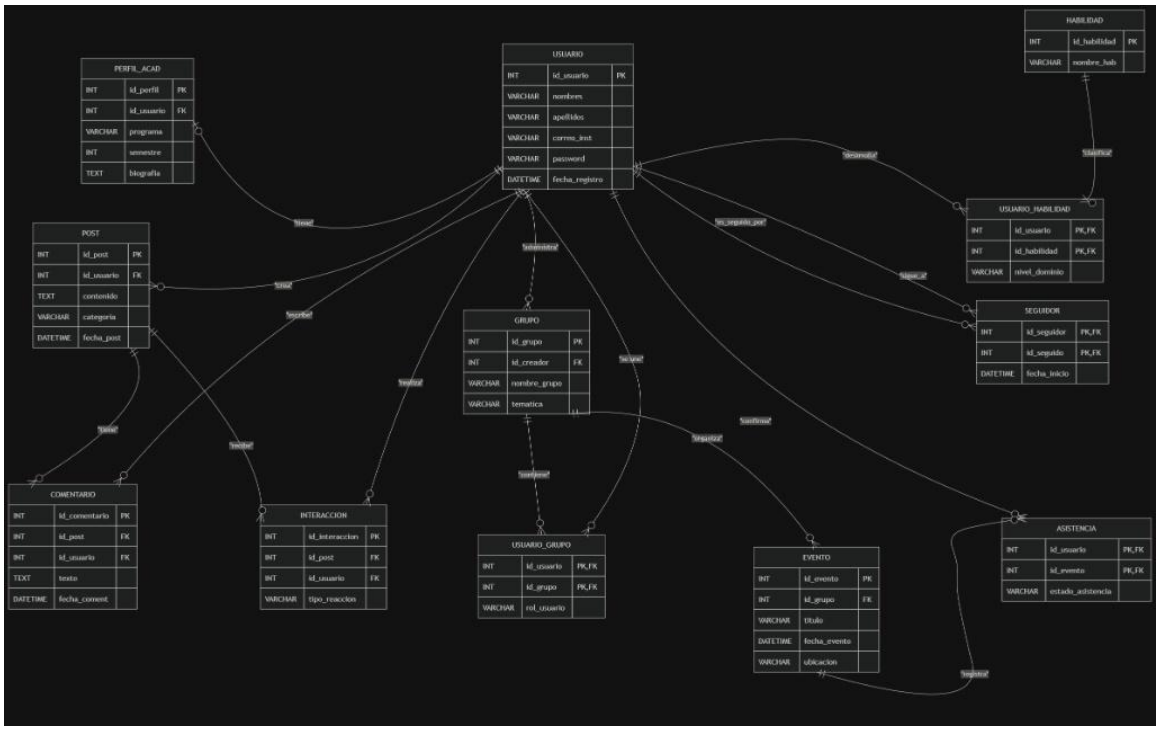
Implementación de la Base de Datos para la Red Social Estudiantil Pascualina

Institución Universitaria Pascual Bravo | **Estudiante: Daniel Grisales**

1. Introducción y Modelo Entidad-Relación

En esta tarea se realiza la implementación física de la base de datos para la red social estudiantil de la Institución Universitaria Pascual Bravo. El sistema permite que los estudiantes creen su perfil académico, registren sus habilidades, sigan a otros compañeros, publiquen actualizaciones con comentarios y reacciones, creen grupos de estudio y programen eventos en el campus.

El diseño parte del diagrama Entidad-Relación normalizado que cuenta con 12 tablas: USUARIO, PERFIL_ACAD, HABILIDAD, USUARIO_HABILIDAD, SEGUIDOR, POST, COMENTARIO, INTERACCION, GRUPO, USUARIO_GRUPO, EVENTO y ASISTENCIA.



2. Justificación de la Elección de los Tres SGBD

Para cumplir con la tarea se eligieron tres Sistemas Gestores de Bases de Datos (SGBD) relacionales muy utilizados:

- **1. PostgreSQL:** Se eligió porque es un gestor gratuito de código abierto muy potente y estable, que además permite guardar datos semiestructurados en formato JSON de forma directa.
- **2. MySQL:** Se eligió porque es el gestor más conocido para páginas y aplicaciones web (incluido en entornos como XAMPP), es fácil de usar y también acepta columnas de tipo JSON y números pequeños como TINYINT.
- **3. Microsoft SQL Server:** Se eligió porque es uno de los gestores principales vistos en el material del curso con la herramienta SQL Server Management Studio (SSMS) y permite comparar cómo cambian las instrucciones SQL frente a PostgreSQL y MySQL.

3. Uso de Tipos de Datos SQL y NoSQL para Big Data e IoT

La guía pide configurar la base de datos para manejar datos semiestructurados seleccionando un tipo NoSQL (como JSON) y optimizar tipos de datos pensando en un caso de uso de Big Data o Internet de las Cosas (IoT):

- **Tipo NoSQL (JSON) para coordenadas geográficas en la tabla EVENTO:** En la tabla EVENTO se agregó el campo coordenadas de tipo JSON. Este campo sirve para guardar la latitud, la longitud, el bloque y el aula del evento en el campus, por ejemplo: {"latitud": 6.2736, "longitud": -75.5880, "bloque": "Bloque 8", "aula": "302"}. Se relaciona con Big Data e IoT porque permite guardar coordenadas de GPS o datos de sensores del salón sin tener que crear muchas columnas nuevas.
- **Optimización del tamaño de dato en SQL (SMALLINT / TINYINT):** En la tabla PERFIL_ACAD, el campo semestre solo toma valores del 1 al 12. En lugar de usar un INT normal que ocupa 4 bytes, se usó TINYINT (1 byte en MySQL y SQL Server) y SMALLINT (2 bytes en PostgreSQL). En aplicaciones de Big Data o sensores IoT que guardan millones de datos, usar enteros pequeños ahorra más de la mitad del espacio en disco.

4. Explicación Paso a Paso de la Implementación y Scripts DDL

Los scripts completos con las instrucciones CREATE TABLE se entregan en la carpeta /tarea4/ddl/ (archivos ddl_postgresql.sql, ddl_mysql.sql y ddl_sqlserver.sql). Los pasos que se siguieron para crearlos fueron:

1. **Paso 1 (Revisar el diccionario genérico):** Se verificaron las 12 tablas del diagrama ER con sus llaves primarias (PK) y llaves foráneas (FK).
2. **Paso 2 (Crear primero las tablas sin llaves foráneas):** En cada script se crearon primero USUARIO y HABILIDAD, ya que no dependen de ninguna otra tabla.
3. **Paso 3 (Ajustar las llaves primarias en cada motor):** En PostgreSQL se definió INT PRIMARY KEY, en MySQL se usó INT AUTO_INCREMENT PRIMARY KEY y en SQL Server se usó INT IDENTITY(1,1) PRIMARY KEY.
4. **Paso 4 (Crear las tablas con llaves foráneas):** Se crearon PERFIL_ACAD, USUARIO_HABILIDAD, SEGUIDOR, POST, COMENTARIO, INTERACCION, GRUPO y USUARIO_GRUPO agregando la instrucción FOREIGN KEY (...) REFERENCES ...
5. **Paso 5 (Crear EVENTO con el campo JSON y ASISTENCIA):** En PostgreSQL y MySQL el campo se puso como coordenadas JSON, mientras que en SQL Server se puso como coordenadas NVARCHAR(MAX) porque SQL Server guarda los JSON dentro de cadenas de texto largas.

Ejemplo del código DDL en la tabla EVENTO en los 3 motores:

```
-- En PostgreSQL (ddl_postgresql.sql) y MySQL (ddl_mysql.sql):
CREATE TABLE EVENTO (
    id_evento INT PRIMARY KEY,
    id_grupo INT NOT NULL,
    titulo VARCHAR(120) NOT NULL,
    fecha_evento TIMESTAMP,
    ubicacion VARCHAR(150),
```

```
    coordenadas JSON,
    FOREIGN KEY (id_grupo) REFERENCES GRUPO(id_grupo)
);

-- En SQL Server (ddl_sqlserver.sql):
CREATE TABLE EVENTO (
    id_evento INT IDENTITY(1,1) PRIMARY KEY,
    id_grupo INT NOT NULL,
    titulo VARCHAR(120) NOT NULL,
    fecha_evento DATETIME,
    ubicacion VARCHAR(150),
    coordenadas NVARCHAR(MAX),
    FOREIGN KEY (id_grupo) REFERENCES GRUPO(id_grupo)
);
```

5. Diccionario de Datos Genérico y Físico de los Tres SGBD

En la siguiente tabla se presenta el Diccionario de Datos Genérico actualizado junto con la equivalencia física en los tres SGBD (también disponible por separado en los archivos Excel de la carpeta /tarea4/diccionarios/):

Tabla	Columna	Genérico	Tam.	Clave	PostgreSQL	MySQL	SQL Server	Descripción
USUARIO	id_usuario	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID del estudiante
USUARIO	nombres	VARCHAR	80	-	VARCHAR(80)	VARCHAR(80)	VARCHAR(80)	Nombres del estudiante
USUARIO	apellidos	VARCHAR	80	-	VARCHAR(80)	VARCHAR(80)	VARCHAR(80)	Apellidos del estudiante
USUARIO	correo_inst	VARCHAR	120	-	VARCHAR(120)	VARCHAR(120)	VARCHAR(120)	Correo institucional
USUARIO	password	VARCHAR	255	-	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)	Contraseña de la cuenta
USUARIO	fecha_registro	DATETIME	19	-	TIMESTAMP	DATETIME	DATETIME	Fecha de registro
PERFIL_ACAD	id_perfil	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID del perfil académico

Tabla	Columna	Genérico	Tam.	Clave	PostgreSQL	MySQL	SQL Server	Descripción
PERFIL_ACAD	id_usuario	INT	10	FK	INT	INT	INT	ID del usuario dueño del perfil
PERFIL_ACAD	programa	VARCHAR	100	-	VARCHAR(100)	VARCHAR(100)	VARCHAR(100)	Carrera que estudia
PERFIL_ACAD	semestre	SMALLINT	2	-	SMALLINT	TINYINT	TINYINT	Semestre actual (entero corto de 1-2 bytes)
PERFIL_ACAD	biografia	TEXT	500	-	TEXT	TEXT	VARCHAR(MAX)	Biografía e intereses del estudiante
HABILIDAD	id_habilidad	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID de la habilidad
HABILIDAD	nombre_hab	VARCHAR	80	-	VARCHAR(80)	VARCHAR(80)	VARCHAR(80)	Nombre de la habilidad o deporte
USUARIO_HABILIDAD	id_usuario	INT	10	PK, FK	INT	INT	INT	ID del estudiante
USUARIO_HABILIDAD	id_habilidad	INT	10	PK, FK	INT	INT	INT	ID de la habilidad
USUARIO_HABILIDAD	nivel_dominio	VARCHAR	30	-	VARCHAR(30)	VARCHAR(30)	VARCHAR(30)	Nivel (Básico, Intermedio, Avanzado)
SEGUIDOR	id_seguidor	INT	10	PK, FK	INT	INT	INT	Usuario que sigue
SEGUIDOR	id_seguido	INT	10	PK, FK	INT	INT	INT	Usuario que es seguido

Tabla	Columna	Genérico	Tam.	Clave	PostgreSQL	MySQL	SQL Server	Descripción
SEGUIDOR	fecha_inicio	DATETIME	19	-	TIMESTAMP	DATETIME	DATETIME	Fecha en que empezó a seguirlo
POST	id_post	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID de la publicación
POST	id_usuario	INT	10	FK	INT	INT	INT	Autor de la publicación
POST	contenido	TEXT	1000	-	TEXT	TEXT	VARCHAR(MAX)	Texto de la publicación
POST	categoria	VARCHAR	50	-	VARCHAR(50)	VARCHAR(50)	VARCHAR(50)	Categoría (Tarea, Recurso, Meme)
POST	fecha_post	DATETIME	19	-	TIMESTAMP	DATETIME	DATETIME	Fecha de la publicación
COMENTARIO	id_comentario	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID del comentario
COMENTARIO	id_post	INT	10	FK	INT	INT	INT	Publicación comentada
COMENTARIO	id_usuario	INT	10	FK	INT	INT	INT	Usuario que comenta
COMENTARIO	texto	TEXT	500	-	TEXT	TEXT	VARCHAR(MAX)	Texto del comentario
COMENTARIO	fecha_coment	DATETIME	19	-	TIMESTAMP	DATETIME	DATETIME	Fecha del comentario
INTERACCION	id_interaccion	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID de la reacción

Tabla	Columna	Genérico	Tam.	Clave	PostgreSQL	MySQL	SQL Server	Descripción
INTERACCION	id_post	INT	10	FK	INT	INT	INT	Publicación a la que reacciona
INTERACCION	id_usuario	INT	10	FK	INT	INT	INT	Usuario que reacciona
INTERACCION	tipo_reaccion	VARCHAR	30	-	VARCHAR(30)	VARCHAR(30)	VARCHAR(30)	Tipo de reacción (Me gusta, etc.)
GRUPO	id_grupo	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID del grupo de estudio
GRUPO	id_creador	INT	10	FK	INT	INT	INT	Usuario creador del grupo
GRUPO	nombre_grupo	VARCHAR	100	-	VARCHAR(100)	VARCHAR(100)	VARCHAR(100)	Nombre del grupo
GRUPO	tematica	VARCHAR	80	-	VARCHAR(80)	VARCHAR(80)	VARCHAR(80)	Tema del grupo
USUARIO_GRUPO	id_usuario	INT	10	PK, FK	INT	INT	INT	Miembro del grupo
USUARIO_GRUPO	id_grupo	INT	10	PK, FK	INT	INT	INT	Grupo al que se une
USUARIO_GRUPO	rol_usuario	VARCHAR	30	-	VARCHAR(30)	VARCHAR(30)	VARCHAR(30)	Rol en el grupo
EVENTO	id_evento	INT	10	PK	INT	INT AUTO_INCREMENT	INT IDENTITY(1,1)	ID del evento
EVENTO	id_grupo	INT	10	FK	INT	INT	INT	Grupo que organiza el evento
EVENTO	titulo	VARCHAR	120	-	VARCHAR(120)	VARCHAR(120)	VARCHAR(120)	Título del evento
EVENTO	fecha_evento	DATETIME	19	-	TIMESTAMP	DATETIME	DATETIME	Fecha y hora del evento

Tabla	Columna	Genérico	Tam.	Clave	PostgreSQL	MySQL	SQL Server	Descripción
EVENTO	ubicacion	VARCHAR	150	-	VARCHAR(150)	VARCHAR(150)	VARCHAR(150)	Lugar o salón del evento
EVENTO	coordenadas	JSON	500	-	JSON	JSON	NVARCHAR(MAX)	[NoSQL Big Data/IoT] Coordenadas geográficas (latitud, longitud) y sensor del salón
ASISTENCIA	id_usuario	INT	10	PK, FK	INT	INT	INT	Usuario que asiste
ASISTENCIA	id_evento	INT	10	PK, FK	INT	INT	INT	Evento al que asiste
ASISTENCIA	estado_asistencia	VARCHAR	30	-	VARCHAR(30)	VARCHAR(30)	VARCHAR(30)	Estado (Confirmado, Pendiente)

6. Dificultades Encontradas, Ventajas y Desventajas

Durante la creación de los scripts en los tres gestores se observaron los siguientes puntos:

- **Dificultades encontradas:** La principal dificultad fue que cada gestor tiene nombres distintos para algunos tipos de datos. Por ejemplo, en SQL Server no existe el tipo TEXT moderno ni el tipo JSON directo, por lo que hubo que cambiarlos por VARCHAR(MAX) y NVARCHAR(MAX). También en PostgreSQL no existe DATETIME ni TINYINT, así que se usaron TIMESTAMP y SMALLINT.
- **Ventajas y desventajas de PostgreSQL:** Tiene como ventaja que soporta el tipo JSON directamente y su código es muy claro; como desventaja, no tiene el tipo TINYINT de 1 byte.
- **Ventajas y desventajas de MySQL:** Es el más fácil de entender y probar en XAMPP porque usa los mismos nombres del diagrama (DATETIME, TEXT, JSON, TINYINT); como desventaja, es menos estricto en algunas validaciones por defecto.
- **Ventajas y desventajas de SQL Server:** Su programa SSMS es muy cómodo para ver las tablas creadas; como desventaja, obliga a usar IDENTITY(1,1) y NVARCHAR(MAX) en vez de JSON.

7. Conclusiones y Propuestas de Mejora

- **Conclusiones del trabajo:** Realizar la traducción de las 12 tablas a PostgreSQL, MySQL y SQL Server permitió aprender en la práctica cómo funciona la instrucción CREATE TABLE en diferentes motores. Además, se comprendió cómo incluir un campo NoSQL (JSON) dentro de una base de datos relacional para guardar coordenadas geográficas sin perder el orden de las tablas.
- **Conclusión individual:** Hacer este ejercicio de manera individual me ayudó a identificar los errores más comunes al escribir scripts SQL (como el orden en que se deben crear las tablas para que no fallen las llaves foráneas) y la importancia de elegir bien el tamaño de los tipos de datos.
- **Propuestas de mejora:** A futuro se podría agregar también un campo JSON en la tabla POST para guardar enlaces de imágenes o archivos adjuntos, y agregar datos de sensores en la tabla ASISTENCIA para registrar la entrada automática a los eventos.